

APPENDIX A
TIME COMPLEXITIES OF VBBMC ALGORITHMS

TABLE VII

THE WORST-CASE TIME COMPLEXITIES OF EXISTING VBBMC ALGORITHMS, WHERE n , h AND δ ARE THE NUMBER OF THE VERTICES, THE h -INDEX AND THE DEGENERACY OF THE GRAPH, RESPECTIVELY.

Algorithm name	Time complexity
BK [28]	$O(n \cdot (3.14)^{n/3})$
BK_Pivot [22], BK_Ref [25]	$O(n \cdot 3^{n/3})$
BK_Degree [29]	$O(hn \cdot 3^{h/3})$
BK_Degen [23], [24]	$O(\delta n \cdot 3^{\delta/3})$
BK_Rcd [13]	$O(\delta n \cdot 2^\delta)$
BK_Fac [30]	$O(\delta n \cdot (3.14)^{\delta/3})$

Different variants of VBBMC have different time complexities, which we summarize in Table VII. We note that BK_Rcd [13] and BK_Fac [30] do not provide the worst-case time complexity analysis in their paper. We provide the analysis as follows.

A. Time Complexity of BK_Rcd

We present the algorithm details of BK_Rcd [13] in Algorithm 9. Each branch is represented by three vertex sets S , C and X (correspondingly, $g_C = G[C]$ and $g_X = G[X]$ are the candidate and exclusion subgraph as introduced in Section III).

Theorem 5. *Given a graph $G = (V, E)$, the worst-case time complexity of BK_Rcd is $O(n\delta \cdot 2^\delta)$, where δ is the degeneracy of the graph.*

Proof. From line 6 and lines 8-9, we observe that, given a branch $B = (S, C, X)$, the algorithm iteratively removes the vertex with the minimum degree in the remaining candidate subgraph $G[C]$. This corresponds to using degeneracy ordering of $G[C]$ for branching. Let $T(c, x)$ be the total time cost to enumerate all maximal cliques within a branch $B = (S, C, X)$, where $|C| = c$ and $|X| = x$. Consider the initial branch. Based on the degeneracy ordering, the candidate subgraph of each produced sub-branch have at most δ vertices. Consider the branches other than the initial branch. While removing the vertex with the minimum degree is intuitive enough such that the remaining graph $G[C]$ is possible to be a non-trivial clique (other than a vertex or an edge) at line 10, it is not always the case. Consider an example as illustrated in Figure 2. The number in each vertex represents the index of the vertex. It is easy to check that the sequence $\{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$ is a valid degeneracy ordering. However, only when C has $\{v_7, v_8\}$ inside, it is a clique (i.e., an edge). All above examples demonstrate that given a branch B , the BK_Rcd would produce $|C| - 2$ branches at the worst case. Thus, we have the following recurrence.

$$T(c, x) \leq \begin{cases} O(\delta) & c = 0 \\ O(|E|) + \sum_{v \in V} T(\delta, \Delta) & c = |V| \\ \sum_{i=2}^{c-1} T(i, x) & c \neq 0 \end{cases} \quad (7)$$

By solving the recurrence, we have the time complexity of BK_Rcd, which is $O(\delta n \cdot 2^\delta)$. $\square$

Algorithm 9: BK_Rcd [13]

Input: A graph $G = (V, E)$
Output: All maximal cliques within G

```

1 BK_Rcd_Rec( $\emptyset, V, \emptyset$ );
2 Procedure BK_Rcd_Rec( $S, C, X$ )
   /* Termination if  $C, X$  are empty */
3   if  $C \cup X = \emptyset$  then
4     | Output a maximal clique  $S$ ; return;
   /* Branching when  $G[C]$  is not a clique */
5   while  $G[C]$  is not a clique do
6     | Choose  $v \in C$  that minimizes  $|N(u, G[C])|$ ;
7     | BK_Rcd_Rec( $S \cup \{v\}, C \cap N(v, G), X \cap N(v, G)$ );
8     |  $C \leftarrow C \setminus \{v\}$ ;
9     |  $X \leftarrow X \cup \{v\}$ ;
   /* Check maximality */
10  if  $C \neq \emptyset$  and  $N(C, G) \cap X = \emptyset$  then
11    | Output a maximal clique  $S \cup C$ ;

```

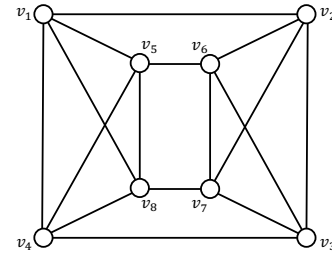


Fig. 2. A worst case for BK_Rcd algorithm.

B. Time Complexity of BK_Fac

We present the algorithm details of BK_Fac [30] in Algorithm 10. Similarly, each branch is represented by three sets S , C and X (correspondingly, $g_C = G[C]$ and $g_X = G[X]$ are the candidate and exclusion subgraph as introduced in Section III).

Theorem 6. *Given a graph $G = (V, E)$, the worst-case time complexity of BK_Fac is $O(\delta n \cdot (3.14)^{\delta/3})$, where δ is the degeneracy of the graph.*

Proof. Based on the degeneracy ordering, each produced sub-branch from line 5 will have at most δ vertices in C_i , where δ is the degeneracy of the graph. Then consider the time complexity of the BK_Fac_Rec. We have the following observations. First, the initial pivot is chosen from C (line 9) and P is the vertex set that includes those vertices at which the branching steps are conducted (line 10). Second, although lines 15-17 would update P to reduce the number of produced branches, there exists the worst case in which P is not changed. This worst case is the same as the second algorithm proposed in [28], whose time complexity is $O(|C| \cdot (3.14)^{|C|/3})$ given a branch $B = (S, C, X)$. In summary, the time complexity of BK_Fac is $O(\delta n \cdot (3.14)^{\delta/3})$. $\square$

Algorithm 10: BK_Fac [30]

Input: A graph $G = (V, E)$
Output: All maximal cliques within G

```

1 Let  $v_1, \dots, v_n$  be in the degeneracy ordering of  $G$ ;
2 for each  $v_i \in V$  do
3    $C_i \leftarrow N(v_i, G) \cap \{v_1, \dots, v_{i-1}\}$ ;
4    $X_i \leftarrow N(v_i, G) \cap \{v_{i+1}, \dots, v_n\}$ ;
5   BK_Fac_Rec( $\{v_i\}, C_i, X_i$ );
6 Procedure BK_Fac_Rec( $S, C, X$ )
   /* Termination if  $C, X$  are empty */
7   if  $C \cup X = \emptyset$  then
8     Output a maximal clique  $S$ ; return;
   /* Initialize a pivot  $v$ , create
   branches on  $P$  */
9    $v \leftarrow$  an arbitrary vertex in  $C$ ;
10   $P \leftarrow C \setminus N(v, G)$ ;
11  for each  $u \in P$  do
12    BK_Fac_Rec( $S \cup \{u\}, C \cap N(u, G), X \cap$ 
13       $N(u, G)$ );
14     $C \leftarrow C \setminus \{u\}$ ;
15     $X \leftarrow X \cup \{u\}$ ;
16    /* Update  $P$  if possible */
17     $P \leftarrow P \setminus \{u\}$ ;
18     $P' \leftarrow C \setminus N(u, G)$ ;
19    if  $|P'| < |P|$  then  $P \leftarrow P'$ ;
```

APPENDIX B
PROOF OF THEOREM 2

Proof. The running time of HBBMC consists of (1) the time of generating the truss-based edge ordering and obtaining the subgraphs (i.e., g_C and g_X) for enumeration, which can be done in $O(\delta m)$ time, where δ is the degeneracy of the graph, and (2) the time of recursive enumeration procedure.

Consider the former. First, the time complexity of generating the truss-based edge ordering is $O(\delta m)$ [31]. Besides, the time of getting the subgraphs can also be bounded by $O(\delta m)$. The reason is as follows. First, for all edges $e = (u, v) \in E$, we can compute the set of common neighbors of u and v (i.e., $N(V(e), G)$ for $e \in E$) in $O(\delta m)$ time by any existing triangle listing algorithm [2], [3], [4]. Second, consider an edge $e = (u, v) \in E$. For each $w \in N(V(e), G)$, $w \in V(g_C)$ if both edges (u, w) and (v, w) are ordered after e ; otherwise, $w \in V(g_X)$. This process can also be done in $O(\delta m)$. To see this, we note that u, v and w always form a triangle (i.e., 3-clique), indicating that the time cost for getting all subgraphs for the sub-branches is proportional to the number of triangles in G . According to the existing triangle listing algorithms [2], [3], [4], given a graph with its degeneracy number equal to δ , the number of triangles inside is bounded by $O(\delta m)$. With $V(g_C)$ and $V(g_X)$, it is sufficient to recursively conduct the vertex-oriented branching strategy for the remaining branches.

Consider the latter, we analyze the time costs for the recursive process. We first show two lemmas that is necessary for the proof.

Lemma 1 (Theorem 3 of [22]). *Let $F(n)$ be a function which satisfies the following recurrence relation*

$$F(n) \leq \begin{cases} \max_k \{k \cdot F(n-k)\} + O(n^2) & \text{if } n > 0 \\ O(1) & \text{if } n = 0 \end{cases} \quad (8)$$

where n and k are positive integers. Then $F(n) = O(3^{n/3})$.

Lemma 2. *Given a graph $G = (V, E)$ with its truss number equal to τ . The number of triangles in G is no more than $m\tau$.*

Proof. Let $\pi_\tau = \langle e_1, e_2, \dots, e_m \rangle$ be the truss-based edge ordering and $\text{sup}(e_i)$ be the number of common neighbors in the subgraph induced by $\{e_i, e_{i+1}, \dots, e_m\}$. Then, the number of triangles in G , denoted by N , is bounded by

$$N = \sum_{i=1}^m \text{sup}(e_i) \leq \sum_{i=1}^m \tau = m\tau \quad (9)$$

The first equation follows from the fact that each triangle is uniquely associated with the earliest edge in the truss-based ordering of its three constituent edges. The inequality follows from the definition of τ . $\square$

Now, we are ready for the analysis. Consider a branch $B = (S, g_C, g_X)$ produced after the edge-oriented branching strategy, let $T(c, x)$ be the time complexity of solving B with $c = |V(g_C)|$ and $x = |V(g_X)|$. The time cost consists of three components. First, it would take $O(c \cdot (c + x))$ to choose the pivot by checking each vertex in $V(g_C) \cup V(g_X)$ to identify the number of its neighbors in g_C . Second, assume that the pivot is v^* and v^* has k ($k \leq c$) non-neighbors in g_C (denoted by $u_1, \dots, u_k$), on which the sub-branches are produced. Correspondingly, for each sub-branch produced on u_i , it would take $O(c + x)$ to identify u_i 's neighbors in g_C and g_X . In total, it would take $O(k \cdot (c + x))$ in total to construct k sub-branches. Third, since v^* is a pivot that has the maximum number of neighbors in g_C , i.e., $c - k$ neighbors, for each $u_i \in V(g_C)$, they all have at most $c - k$ neighbors in g_C . Thus, we have the following recurrence

$$\begin{aligned} T(c, x) &\leq \max_k \{k \cdot T(c - k, x)\} + \lambda \cdot c \cdot (c + x) \\ &= (c + x) \cdot \left(\max_k \left\{ k \cdot \frac{T(c - k, x)}{c + x} \right\} + \lambda \cdot c \right) \\ &\leq (c + x) \cdot \left(\max_k \left\{ k \cdot \frac{T(c - k, x)}{c - k + x} \right\} + \lambda \cdot c^2 \right) \end{aligned} \quad (10)$$

The first inequality follows by summing up the three components of the time cost, where $\lambda > 0$ is a constant. The equality follows by extracting $(c + x)$. To see the third inequality, we first have $\lambda \cdot c \leq \lambda \cdot c^2$ since $c \geq 1$. Then, let k^* be value of k such that $k \cdot \frac{T(c - k, x)}{c + x}$ reaches the maximum. We have

$$\begin{aligned} \max_k \left\{ k \cdot \frac{T(c - k, x)}{c + x} \right\} &= k^* \cdot \frac{T(c - k^*, x)}{c + x} \\ &\leq k^* \cdot \frac{T(c - k^*, x)}{c - k^* + x} \\ &\leq \max_k \left\{ k \cdot \frac{T(c - k, x)}{c - k + x} \right\} \end{aligned} \quad (11)$$

Let $f(c) = \frac{T(c,x)}{(c+x)}$, we have

$$f(c) \leq \max_k \{k \cdot f(c-k)\} + \lambda \cdot c^2 \quad (12)$$

According to Lemma 1, we have $f(c) = \lambda' \cdot 3^{c/3}$, where $\lambda' > 0$ is also a constant. Therefore, $T(c, x) = \lambda' \cdot (c + x) \cdot 3^{c/3}$. To compute the total time complexity,

$$\begin{aligned} \sum_{e \in E} T(c_e, x_e) &= \sum_{e \in E} \lambda' \cdot (c_e + x_e) \cdot 3^{c_e/3} \\ &\leq \lambda' \cdot 3^{\tau/3} \cdot \sum_{e \in E} (c_e + x_e) \leq \lambda' \cdot 3^{\tau/3} \cdot 3m\tau \end{aligned} \quad (13)$$

The first inequality follows from the fact that $c_e \leq \tau$ based on the definition of τ . The second inequality follows from the facts that (1) for an edge e , $(c_e + x_e)$ is the number of triangles associated with e , and (2) the total number of triangles is no larger than $m\tau$ (Lemma 2).

In summary the total time cost of HBBMC is $O(m\delta + m\tau \cdot 3^{\tau/3})$. $\square$

APPENDIX C

EXAMPLES OF ENUMERATING MAXIMAL CLIQUES FROM 2-PLEXES AND 3-PLEXES

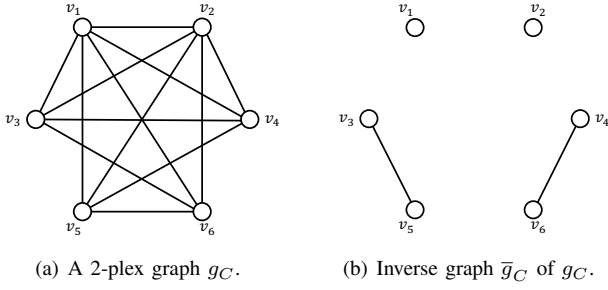


Fig. 3. Example of a 2-plex and its inverse graph.

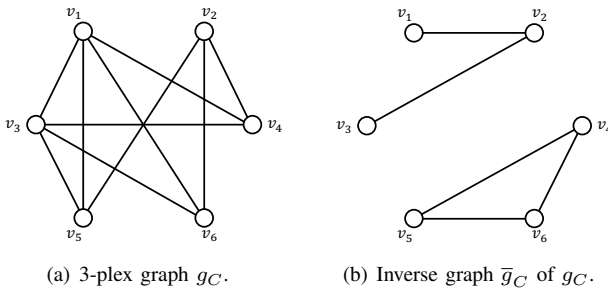


Fig. 4. Example of a 3-plex and its inverse graph.

Example. Consider the example of a 2-plex and its inverse graph in Figure 3. Based on the definition of F , L and R as discussed above, one possible partition is $F = \{v_1, v_2\}$, $L = \{v_3, v_4\}$ and $R = \{v_5, v_6\}$. Given $|L| = 2$, we can directly output 4 maximal cliques as follows, $\{v_1, v_2, v_3, v_4\}$, $\{v_1, v_2, v_3, v_6\}$, $\{v_1, v_2, v_4, v_5\}$ and $\{v_1, v_2, v_5, v_6\}$.

Consider the example of a 3-plex and its inverse graph in Figure 4. Based on the definitions, the inverse graph contains a path $p = \{v_1, v_2, v_3\}$ and a cycle $c = \{v_4, v_5, v_6\}$. Consider

the path p . It has two partial maximal cliques inside, i.e., $\{v_1, v_3\}$ and $\{v_2\}$. Consider the cycle c . It has three partial cliques inside, i.e., $\{v_4\}$, $\{v_5\}$ and $\{v_6\}$. Therefore, there are 6 maximal cliques in g_C , i.e., $\{v_1, v_3, v_4\}$, $\{v_1, v_3, v_5\}$, $\{v_1, v_3, v_6\}$, $\{v_2, v_4\}$, $\{v_2, v_5\}$ and $\{v_2, v_6\}$.

APPENDIX D

EXPERIMENTAL SETUP AND RESULTS ON SYNTHETIC DATASETS

Setup. The synthetic datasets are generated based on two random graph generators, namely the Erdős-Rényi (ER) graph model [42] and the Barabási-Albert (BA) graph model [5]. Specifically, in the ER generator, the model first generates n vertices and then randomly chooses m edges between pairs of vertices. In the BA generator, the model adds the vertices to the network one at a time, and each new vertex is connected to a set number of existing vertices based on preferential attachment, i.e., vertices with a higher degree are more likely to receive new links. By default, the number of vertices and edge density are set as $n = 1,000,000$ and $\rho = 20$ (i.e., $m = n\rho$) for synthetic datasets, respectively. Due to the random nature of the graph generators, given a setting, we generate 10 graphs and report the mean of the results over these graphs.

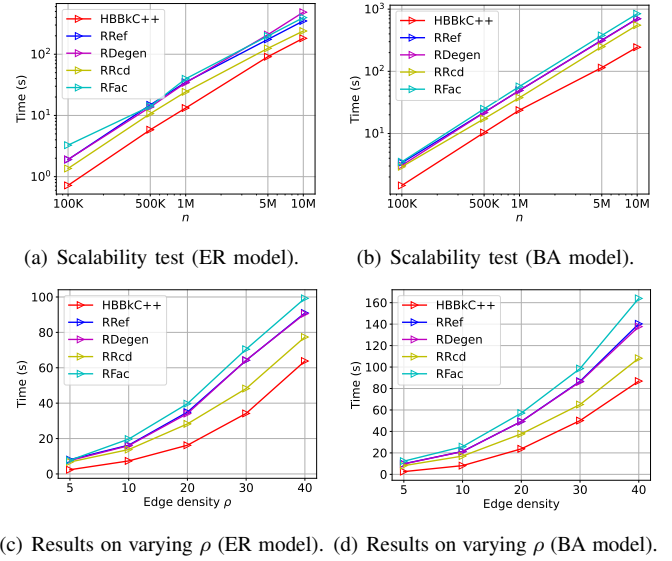


Fig. 5. Results on synthetic datasets.

Results. First, we vary the number of vertices in the range $n \in \{100K, 500K, 1M, 5M, 10M\}$ and set $\rho = 20$ by default. The results for the graphs generated using the ER model and the BA model are presented in Figure 5(a) and Figure 5(b), respectively. Under all settings, HBBMC++ consistently outperforms all baselines, achieving remarkable speedups compared to the second-best algorithm RRcd, with average speedups of 1.84x and 1.94x (up to 2.84x and 2.26x) on the graphs generated by the ER and BA models, respectively. Additionally, we report the degeneracy number δ and the truss-related

R103

R103

number τ under different settings. In the graphs generated by the ER model, these values are $\delta = \{8, 14, 28, 43, 58\}$ and $\tau = \{0, 1, 3, 5, 7\}$ for different values of n . In contrast, for the graphs generated by the BA model, the values remain approximately $\delta \approx 42$ and $\tau \approx 25$ across all n . This interesting phenomenon can be explained by the distinct mechanisms of edge formation in the two models. In the ER model, edges are uniformly distributed, which facilitates the formation of larger k -cores as the number of vertices n increases, leading to an increase in degeneracy. Conversely, in the BA model, the preferential attachment mechanism concentrates edges around a few high-degree hubs, stabilizing the graph's core structure and maintaining a nearly constant degeneracy, irrespective of the growth in n . Finally, we note that all above results are well aligned with our theoretical analysis, i.e., the worst-case time complexity of HBBMC++ is better than that of existing algorithms when $\delta \geq \tau + \frac{3}{\ln 3} \ln \rho$.

Second, we vary the edge density in the range $\rho \in$

$\{5, 10, 20, 30, 40\}$ and set $n = 1\text{M}$ by default. The results for the graphs generated using the ER and BA models are presented in Figure 5(c) and Figure 5(d), respectively. On average, HBBMC++ achieves speedups of 1.65x and 1.88x (with maximum speedups of 1.89x and 3.17x) on the graphs generated by the ER and BA models, respectively, compared to the second-best algorithm RRcd. We also observe that the speedup decreases as the graph ρ increases. The possible reason is that when ρ increases, the graph becomes denser. Therefore, the differences between δ and τ becomes narrower.

Notably, we observe that when comparing the results under identical parameter settings but using the ER model, the computation time is longer for graphs generated by the BA model. The possible reason is that the graphs following a power-law distribution typically contain larger sizes of maximal cliques. For example, under the default setting (i.e., $n = 1\text{M}$ and $\rho = 20$), the largest clique in the ER graph is a 5-clique, while for the BA graph, the largest one is a 28-clique.

R1O3

R1O3